

HireMind AI

Multi-Agent Resume Screening & Job Matching Platform

PROJECT REPORT — Hackathon Problem Statement PS03

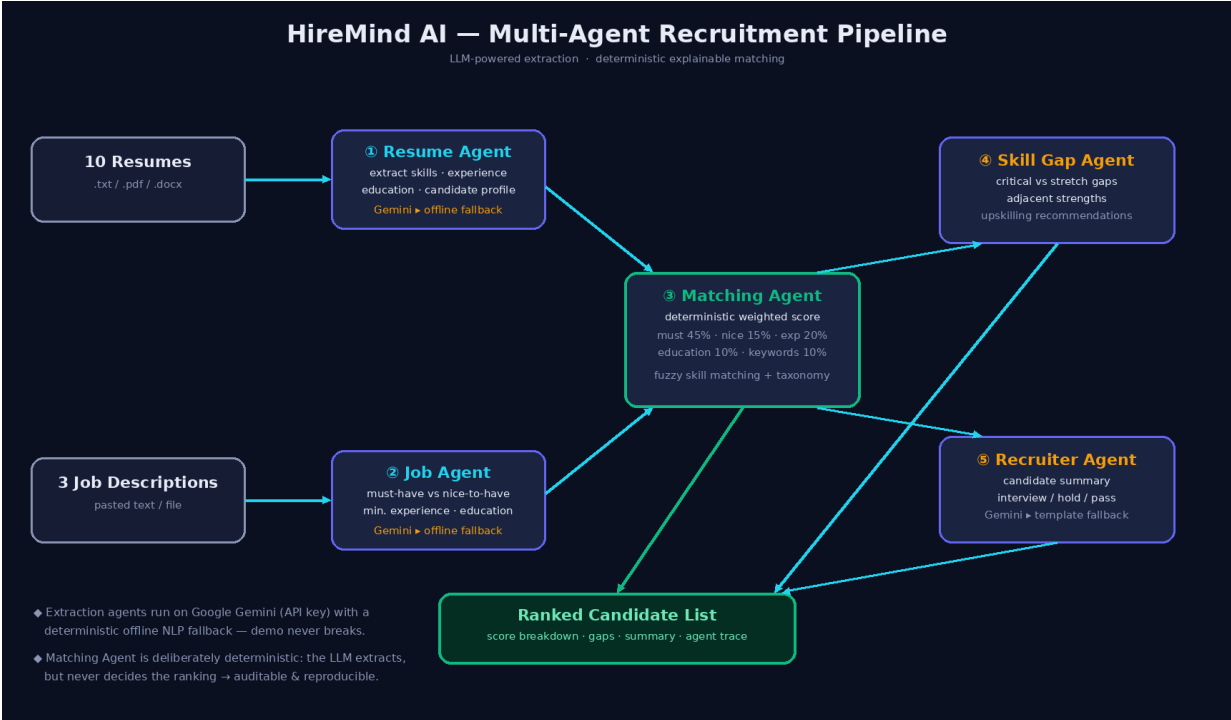


Figure 0 — Five-agent pipeline: LLM-powered extraction, deterministic explainable matching

Field	Details
Problem Statement	PS03 — Multi-Agent Resume Screening & Job Matching Platform
Technology Category	NLP + LLM + Agentic AI
Team	[YOUR TEAM NAME] — [MEMBERS]
Hackathon	[HACKATHON NAME]
Date / Version	20 Sept 2026 · v1.0

1. Executive Summary

2. Problem Analysis

3. Research

3.1 Existing approaches surveyed

3.2 Design conclusions

3.3 Open datasets

4. Proposed Solution

#	Agent	Output	Engine
1	Resume Agent	Candidate profile: canonical skills, experience, education, highlights	Gemini JSON-mode / offline heuristics
2	Job Agent	Must-have vs nice-to-have skills, min experience, education	Gemini JSON-mode / offline heuristics
3	Matching Agent	0–100 score, verdict, per-dimension breakdown, matched/missing skills	Deterministic (always)
4	Skill Gap Agent	Critical & stretch gaps, adjacent strengths, upskilling plan	Deterministic rules
5	Recruiter Agent	Recruiter-style summary + interview / hold / pass recommendation	Gemini / template

5. Technical Flow & Scoring Model

5.1 Scoring model

Dimension	Weight	Computation
Must-have skills	45%	Fraction of required must-haves matched (fuzzy)
Nice-to-have skills	15%	Fraction matched
Experience	20%	1.0 if candidate $\geq$ required years; $0.6 \times (\text{cand}/\text{req})$ below; 0.75 if unspecified
Education	10%	Degree-rank comparison (PhD > Master > Bachelor > Diploma)
Keyword affinity	10%	JD keywords / titles found in resume text

5.2 REST API

Endpoint	Purpose
POST /api/resumes/[upload] · POST /api/jobs	Register documents → Agents 1 & 2 run
POST /api/demo/load	Load bundled 10-resume / 3-JD demo set
POST /api/match {job_id}	Agents 3–5 over all candidates → ranked list + traces
POST /api/reextract · GET /api/agents · /api/settings	Re-extraction after adding a key · metadata · LLM config

6. Technology Stack

Layer	Technology	Rationale
Frontend	React 18 + Vite	Agent cards, ranked lists, explainable detail panels; fast HMR
Backend	Python 3 + FastAPI (Uvicorn)	Async REST API, OpenAPI docs, serves the built React app
LLM	Google Gemini 2.5 Flash (REST)	JSON-mode extraction & summarisation; user-supplied key
Offline NLP	Regex + 130-skill taxonomy + difflib	Deterministic fallback, alias normalisation
Parsing	pypdf · python-docx	Real-world resume formats
State	In-memory store	Zero-setup demo; repository pattern ready for SQLite/Postgres

6.1 Repository layout

```
backend/ main.py · store.py · skills_taxonomy.py · offline_extract.py ·
agents/{llm,resume_agent,job_agent,matching_agent,skill_gap_agent,recruiter_agent}.py · static/
(React build) – frontend/ React+Vite source – data/resumes (10) · data/jobs (3) – docs/ report,
deck, diagram, screenshots – test_pipeline.py
```

7. Implementation — Application Screenshots

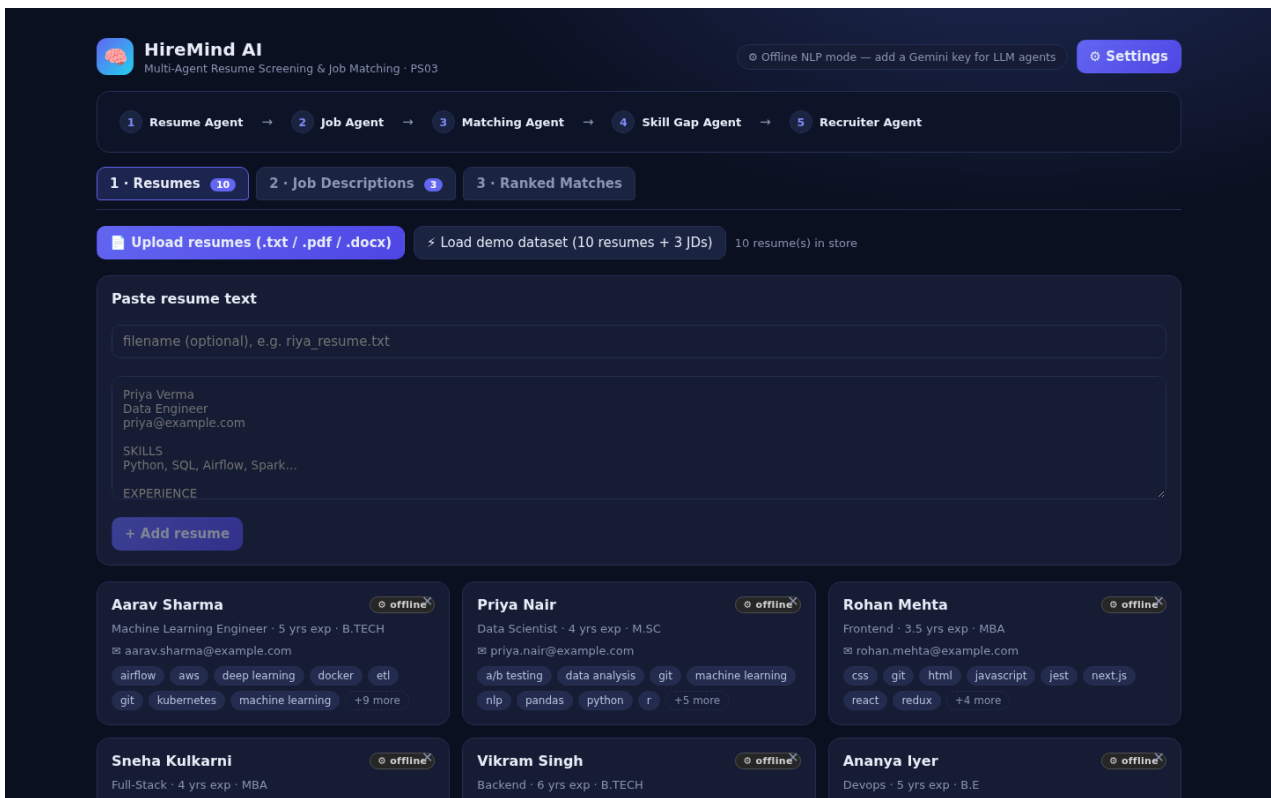


Figure 1 — Resume Agent output: 10 extracted candidate cards (skills, experience, education, engine badge).

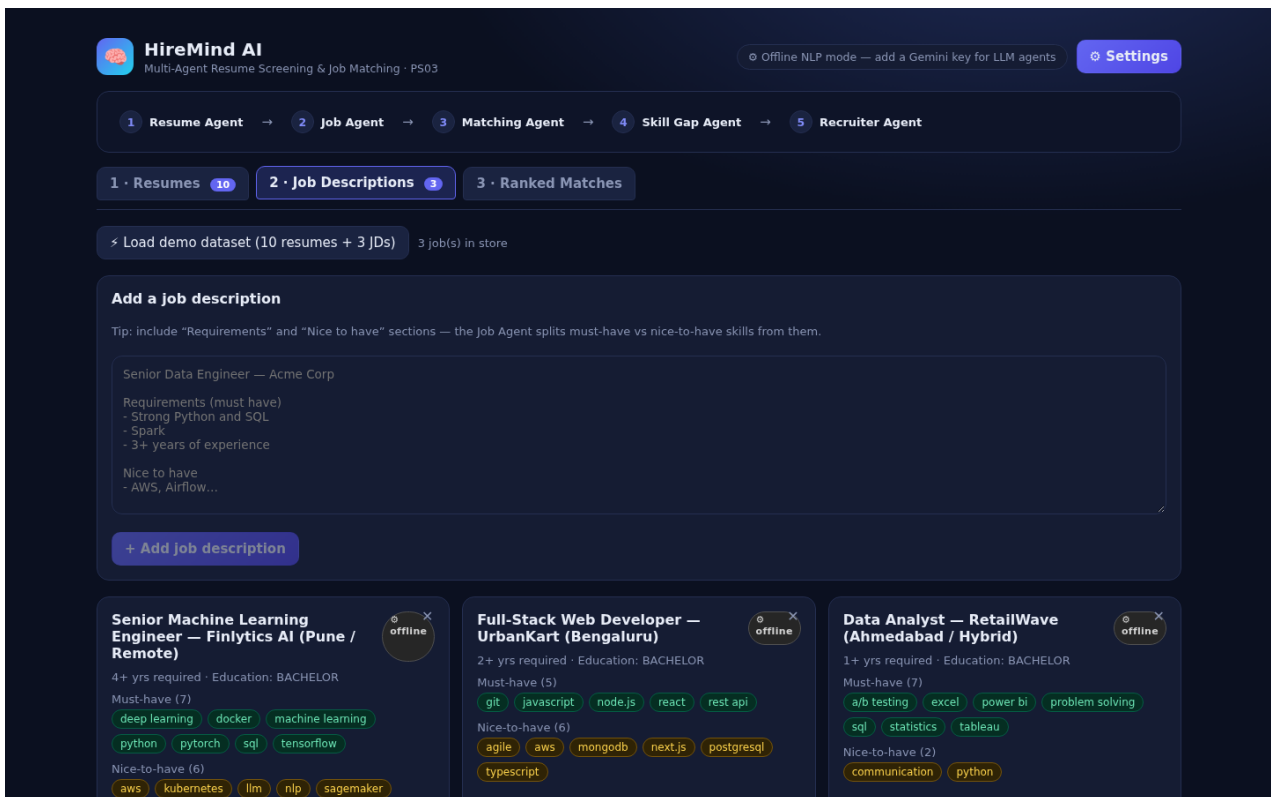


Figure 2 — Job Agent output: must-have (green) vs nice-to-have (amber) requirement chips per JD.

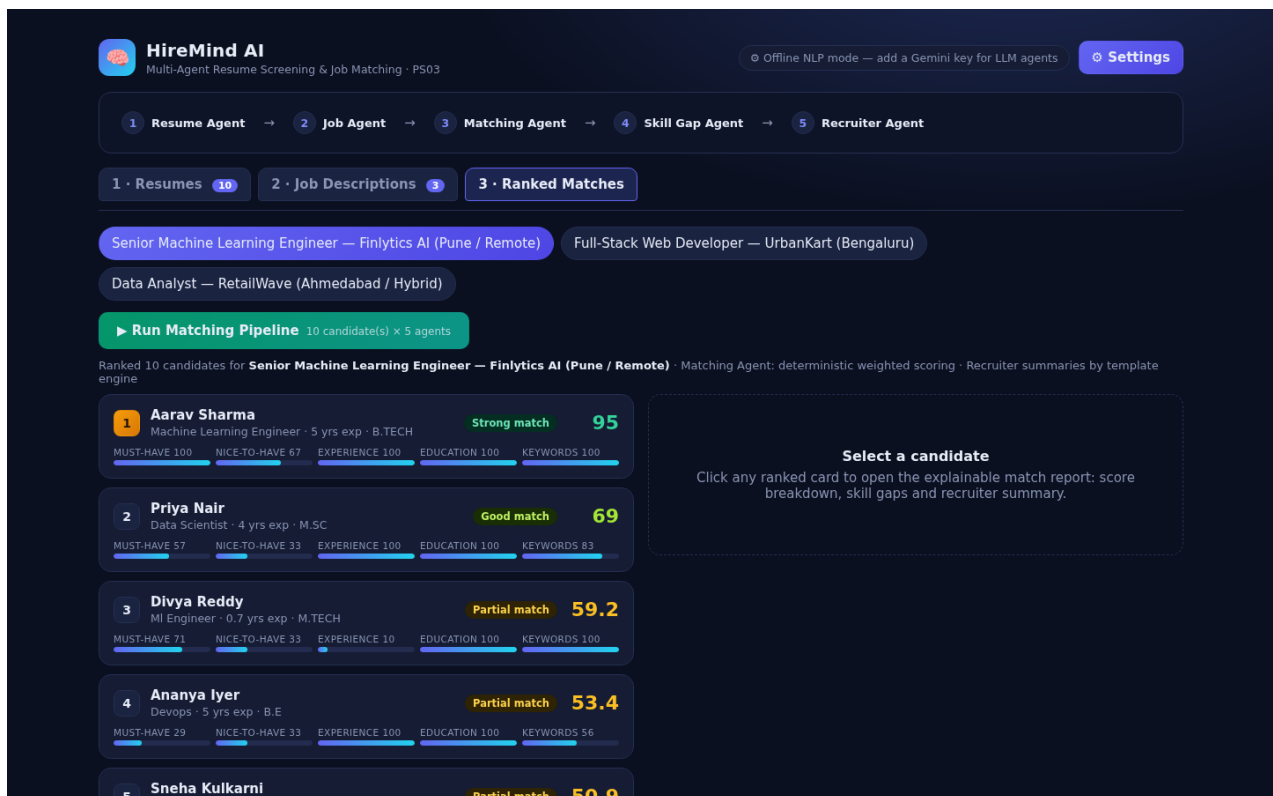


Figure 3 — Ranked candidate list with verdict badges and per-dimension score bars for the Senior ML Engineer role.

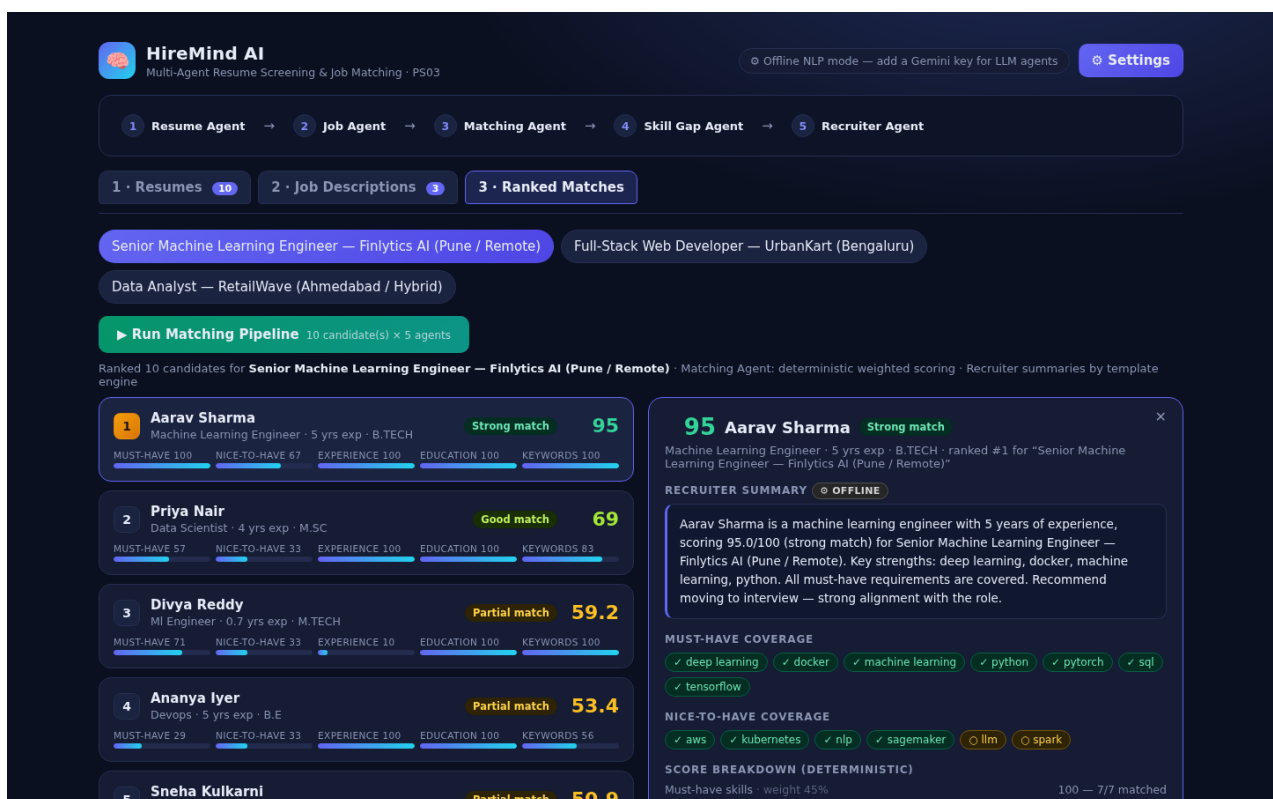


Figure 4 — Explainable match report: recruiter summary, ✓/✗ must-have coverage, nice-to-have coverage, weighted score breakdown.

8. Verified Demonstration Results

Job	#1	#2	#3
Senior ML Engineer	Aarav Sharma — 95.0 Strong (7/7 must)	Priya Nair — 69.0 Good	Divya Reddy — 59.2 Partial
Full-Stack Developer	Sneha Kulkarni — 100.0 Strong (5/5)	Rohan Mehta — 81.0 Strong	Vikram Singh — 66.2 Good
Data Analyst	Karan Patel — 80.7 Strong	Priya Nair — 78.0 Good	Aarav Sharma — 46.7 Partial

9. Explainability, Fairness & Innovation

9.1 Explainability & fairness

9.2 Innovation highlights

10. Limitations & Future Work

11. Use of External / AI Resources (Disclosure)

12. References

Appendix A — Run instructions

```
cd backend && pip install -r requirements.txt && uvicorn main:app --host 0.0.0.0 --port 8000 # serves API + React build · cd frontend && npm install && npm run dev # hot-reload UI, /api proxied · python3 test_pipeline.py # end-to-end smoke test · Open http://localhost:8000 → Load demo dataset → Tab 3 → Run Matching Pipeline. Optionally add a Gemini key under ■ Settings.
```